

UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas
Informáticos



UNIVERSIDAD
POLITÉCNICA
DE MADRID



Máster
Deep Learning
Universidad Politécnica de Madrid

Trabajo Fin de Máster

Sistema multiagente para análisis financiero histórico
mediante generación automática de código

MÁSTER DE FORMACIÓN PERMANENTE
EN DEEP LEARNING

Presentado por:

Carlos Ruiz Oyarzun

Bajo la dirección de:

Adrián Girón Jiménez

Alejandro Delgado Peribáñez

Madrid, 2026

Resumen

Este Trabajo Fin de Máster propone el desarrollo incremental de un sistema multiagente orientado al análisis financiero histórico. El sistema parte de consultas ya normalizadas, datos de mercado almacenados en CSV y un conjunto cerrado de intenciones analíticas. A partir de esa entrada, el prototipo valida la solicitud, selecciona una familia de agente, genera un plan de análisis, construye código Python ejecutable, lanza dicho código en un proceso controlado y transforma la salida en una respuesta interpretable. La arquitectura se ha construido de forma gradual, manteniendo una base determinista y trazable antes de incorporar de forma controlada modelos de lenguaje mediante APIs compatibles con OpenAI.

El MVP actual soporta seis intenciones analíticas: crecimiento de precio, comparación de activos, visión descriptiva de un activo, análisis de retornos, análisis de riesgo histórico y análisis técnico. La evaluación combina pruebas funcionales, escenarios de error controlado y una comparativa entre el modo determinista y varios proveedores LLM. Los resultados muestran que el sistema puede mantener el cálculo financiero bajo control programático y utilizar modelos de lenguaje como una capa de interpretación final para mejorar la redacción sin alterar las métricas calculadas.

Palabras clave: sistema multiagente, análisis financiero, modelos de lenguaje, generación de código, reproducibilidad, fallback determinista.

Abstract

This Master's Thesis proposes the incremental development of a multi-agent system for historical financial analysis. The system starts from already normalized queries, market data stored in CSV files and a closed set of analytical intents. From this input, the prototype validates the request, selects an agent family, generates an analysis plan, builds executable Python code, runs that code in a controlled process and transforms the output into an interpretable response. The architecture has been built gradually, preserving a deterministic and traceable base before incorporating optional calls to language models through APIs compatible with OpenAI.

The current MVP supports six analytical intents: price growth, asset comparison, descriptive view of an asset, returns analysis, historical risk analysis and technical analysis. The evaluation combines functional tests, controlled error scenarios and a comparison between the deterministic mode and several LLM providers. The results show that the system can keep financial computation under programmatic control while using language models only as an optional layer to improve the final wording of the response.

Keywords: multi-agent system, financial analysis, language models, code generation, reproducibility, deterministic fallback.

Índice general

Resumen	I
Abstract	II
1. Introducción	1
1.1. Contexto y definición del problema	1
1.2. Motivación	2
1.3. Objetivos	3
1.4. Alcance	3
1.5. Estructura de la memoria	3
2. Estado de la cuestión	5
2.1. Enfoque de la revisión	5
2.2. Modelos de lenguaje aplicados al dominio financiero	5
2.3. Agentes y sistemas multiagente en finanzas	6
2.4. Agentes con herramientas, planificación y ejecución	6
2.5. Generación de código y análisis de datos	7
2.6. Fiabilidad, alucinaciones y trazabilidad	8
2.7. Comparativa con trabajos relacionados	8
2.8. Posicionamiento de la propuesta	9
3. Datos y material de trabajo	10
3.1. Fuente de datos y trazabilidad	10
3.2. Catálogo de consultas, activos y formato de entrada	10
3.3. Estructura de los CSV	11
3.4. Análisis exploratorio de datos	12
3.5. Calidad, cobertura y limitaciones	12
3.6. Papel de los cuadernos auxiliares	13
4. Metodología	14
4.1. Enfoque incremental del desarrollo	14
4.2. Definición de intenciones analíticas	15

4.3.	Diseño experimental	15
4.4.	Métricas de validación	15
4.5.	Justificación de no entrenar un modelo propio	16
4.6.	Gestión de riesgos técnicos	16
5.	Diseño del sistema	17
5.1.	Arquitectura general	17
5.2.	Entrada estructurada y estado compartido	17
5.3.	Nodos del flujo y comunicación entre componentes	17
5.4.	Familias de agentes analíticos	19
5.5.	Generación controlada de código	19
5.6.	Ejecución segura y artefactos	19
5.7.	Gestión de errores y fallback determinista	20
6.	Implementación	21
6.1.	Estructura del repositorio	21
6.2.	Módulos principales	21
6.3.	Ejecución, despliegue local y reproducibilidad	22
6.4.	Tests automatizados	22
6.5.	Evaluación automática de perfiles LLM	22
6.6.	Artefactos generados	23
7.	Integración controlada de modelos de lenguaje	24
7.1.	Motivación de la capa LLM	24
7.2.	Cliente compatible con OpenAI	24
7.3.	Activación granular	25
7.4.	Configuraciones evaluadas	25
7.5.	Estrategia de fallback	25
7.6.	Observaciones técnicas de la integración	26
7.7.	Gestión de credenciales	26
8.	Resultados y evaluación	27
8.1.	Protocolo de evaluación	27
8.2.	Resultados cuantitativos del modo determinista	28
8.3.	Evaluación por familias de agentes	29
8.4.	Comparación entre enfoque determinista y uso de LLM	30
8.5.	Análisis cualitativo de las propuestas	30
8.6.	Análisis de robustez y límites	34
8.7.	Por qué no basta con una plantilla de código	34
8.8.	Limitaciones de la evaluación	35

8.9. Síntesis	35
9. Aspectos éticos, seguridad y uso responsable	36
9.1. Delimitación financiera del sistema	36
9.2. Riesgos de interpretación	36
9.3. Seguridad en la generación y ejecución de código	36
9.4. Privacidad y datos	37
9.5. Sostenibilidad, costes y recursos	37
9.6. Limitaciones de uso	37
10. Conclusiones y trabajo futuro	38
10.1. Cumplimiento de objetivos	38
10.2. Aportaciones principales	38
10.3. Limitaciones actuales	39
10.4. Líneas de trabajo futuro	39
Referencias	40
A. Anexos	43
A.1. Ejecución del MVP	43
A.2. Configuración de APIs	43
A.3. Documentación interna de apoyo	44

Índice de tablas

2.1. Comparativa entre trabajos relacionados y la propuesta del TFM. . . .	9
3.1. Estructura semántica de una consulta financiera antes de su procesamiento.	11
3.2. Tipos de activos representados en el conjunto de datos.	11
3.3. Resumen de cobertura y calidad del conjunto de datos.	12
3.4. Métricas exploratorias por serie utilizadas para contextualizar los casos de prueba.	12
4.1. Evolución incremental del MVP.	14
4.2. Intenciones analíticas soportadas por el MVP.	15
6.1. Principales módulos del repositorio.	21
8.1. Evaluación cuantitativa del flujo determinista ampliado.	28
8.2. Resumen de agentes, métricas y flujo de interpretación.	29
8.3. Comparación entre modo determinista y perfiles LLM en la fase de in- terpretación.	31
8.4. Análisis cualitativo de las propuestas evaluadas.	32
8.5. Análisis cualitativo por tipo de consulta evaluada.	33
8.6. Familias de casos de estrés utilizados para intentar romper el MVP. . .	34

Índice de figuras

1.1. Mapa conceptual de relaciones entre inteligencia artificial, aprendizaje automático, redes neuronales, aprendizaje profundo e inteligencia artificial generativa. Fuente: Analytics Vidhya [1].	2
5.1. Arquitectura de nodos del flujo multiagente y comunicación mediante estado compartido.	18

Capítulo 1

Introducción

1.1. Contexto y definición del problema

El análisis financiero histórico suele combinar varias tareas que, aunque son conceptualmente distintas, aparecen juntas en el trabajo práctico: carga de datos, validación de entradas, selección del tipo de análisis, cálculo de métricas, generación de gráficos, interpretación de resultados y comunicación final al usuario. En muchos prototipos basados en modelos de lenguaje, estas fases se mezclan dentro de una única llamada generativa, lo que dificulta la trazabilidad del resultado y aumenta el riesgo de errores silenciosos.

Este trabajo plantea una aproximación diferente. En lugar de delegar todo el proceso en un modelo generativo, se diseña un flujo multiagente en el que cada fase tiene una responsabilidad delimitada. El sistema no intenta resolver desde cero el problema completo de comprensión del lenguaje natural, sino que parte de una consulta financiera ya estructurada: intención analítica, símbolos financieros, rango temporal, intervalo y datos históricos asociados. A partir de esta entrada, el objetivo es orquestar un análisis reproducible y controlado.

Para situar el alcance tecnológico del trabajo, la Figura 1.1 resume la relación entre inteligencia artificial, aprendizaje automático, aprendizaje profundo e inteligencia artificial generativa. El TFM no se centra en entrenar modelos ni en desarrollar nuevas arquitecturas neuronales, sino en integrar modelos de lenguaje como capa auxiliar dentro de un sistema trazable de análisis financiero histórico.

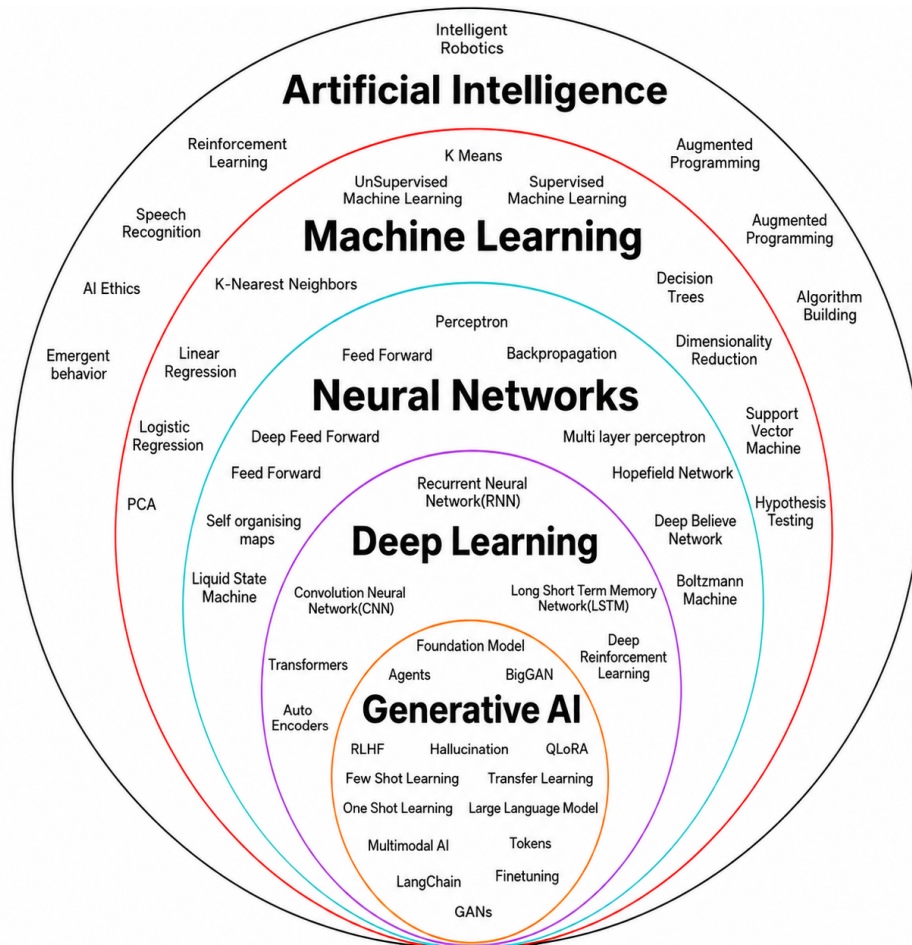


Figura 1.1: Mapa conceptual de relaciones entre inteligencia artificial, aprendizaje automático, redes neuronales, aprendizaje profundo e inteligencia artificial generativa. Fuente: Analytics Vidhya [1].

1.2. Motivación

La motivación principal del proyecto es explorar cómo una arquitectura multiagente puede aportar orden, modularidad y robustez a un sistema de análisis financiero asistido por inteligencia artificial. La aplicación de modelos de lenguaje al dominio financiero resulta atractiva por su capacidad de generar explicaciones y adaptar el lenguaje al usuario, pero también exige mecanismos de control. En un contexto financiero, incluso cuando el objetivo es puramente descriptivo, resulta esencial separar los cálculos verificables de la redacción final.

Por esta razón, el MVP se ha construido inicialmente con plantillas deterministas. Esta decisión permite validar el flujo completo antes de introducir APIs externas o modelos servidos localmente. Una vez estabilizada la arquitectura, la integración de modelos de lenguaje se realiza de forma gradual: primero se prueban APIs externas como Groq, con el modelo llama-3.3-70b-versatile [2], [3], [4], y Gemini, con el

modelo `gemini-2.5-flash` [5], [6]; después se habilita un servidor vLLM compatible con OpenAI Chat Completions usando el modelo `openai/gpt-oss-20b` [7]. En todos los casos, la activación empieza por la fase de interpretación y conserva fallbacks programáticos.

1.3. Objetivos

El objetivo general del trabajo es desarrollar y validar un prototipo multiagente capaz de ejecutar análisis financieros históricos a partir de entradas estructuradas y datos en CSV.

Los objetivos específicos son:

- Definir un flujo modular que separe ingesta, enrutado, planificación, generación de código, ejecución e interpretación.
- Implementar familias de agentes especializadas para distintas intenciones analíticas.
- Mantener una base determinista que permita pruebas reproducibles y control de errores.
- Preparar la arquitectura para integrar modelos de lenguaje mediante APIs compatibles con OpenAI.
- Evaluar el MVP mediante casos funcionales y escenarios de error controlado.

1.4. Alcance

El sistema se limita al análisis histórico-descriptivo de activos financieros a partir de datos ya descargados. No realiza predicción futura, recomendación de inversión ni análisis fundamental complejo. Esta restricción es deliberada: el foco del TFM no es construir un asesor financiero, sino estudiar la arquitectura de un flujo multiagente trazable para análisis histórico.

1.5. Estructura de la memoria

La memoria se organiza siguiendo la lógica de desarrollo del proyecto. En primer lugar se presenta el estado de la cuestión, con especial atención a modelos de lenguaje, arquitecturas multiagente y limitaciones de enfoques monolíticos. A continuación se describen los datos utilizados, su trazabilidad y su calidad. Después se expone la metodología incremental seguida durante el desarrollo.

Los capítulos centrales describen el diseño del sistema, la implementación del MVP y la integración controlada de modelos de lenguaje. Posteriormente se presentan los resultados de evaluación, incluyendo el modo determinista y la comparativa con proveedores LLM. Finalmente se discuten los aspectos éticos y de seguridad, se resumen las conclusiones y se plantean líneas de trabajo futuro.

Capítulo 2

Estado de la cuestión

2.1. Enfoque de la revisión

El objetivo de este capítulo es situar el TFM dentro de tres líneas de trabajo relacionadas: el uso de modelos de lenguaje en finanzas, las arquitecturas de agentes con herramientas externas y la generación controlada de código para análisis de datos. La revisión no pretende cubrir todo el campo de la inteligencia artificial financiera, sino identificar trabajos representativos que permitan comparar la propuesta desarrollada.

El sistema de este TFM se diferencia de una aplicación puramente generativa porque mantiene el cálculo financiero en un flujo programático y utiliza los modelos de lenguaje como una capa auxiliar de interpretación. Por ello, resultan especialmente relevantes los trabajos que combinan lenguaje natural, datos financieros, agentes, uso de herramientas y mecanismos de control.

2.2. Modelos de lenguaje aplicados al dominio financiero

La aplicación de técnicas de procesamiento del lenguaje natural al dominio financiero cuenta con antecedentes previos a los modelos generativos actuales. Un ejemplo representativo es FinBERT, que adapta modelos tipo BERT al análisis de sentimiento financiero y muestra la utilidad de especializar modelos lingüísticos en textos de mercado [8]. Este tipo de trabajo es relevante porque evidencia que el lenguaje financiero tiene vocabulario, polaridad y contexto propios, por lo que no siempre basta con aplicar modelos generales sin adaptación.

Con la aparición de los modelos de lenguaje de gran escala, la investigación se ha desplazado hacia sistemas capaces de operar sobre tareas financieras más amplias. BloombergGPT propone un modelo de 50.000 millones de parámetros entrenado con una mezcla de datos financieros y generales, y evalúa su rendimiento tanto en tareas

financieras como en bancos de prueba generales [9]. FinGPT, por su parte, plantea una alternativa abierta y centrada en la construcción de recursos reutilizables para modelos financieros, destacando la importancia de los flujos de datos y de la adaptación ligera de modelos [10].

Estos trabajos se sitúan en una escala distinta a la de este TFM: su foco principal es el entrenamiento o adaptación de modelos para finanzas, mientras que este proyecto no entrena un modelo financiero propio. La aportación aquí es arquitectónica y metodológica: usar modelos ya disponibles como una capa de redacción sobre cálculos financieros trazables, evitando que el modelo sea responsable directo de las métricas.

2.3. Agentes y sistemas multiagente en finanzas

Los sistemas recientes basados en agentes intentan ir más allá de una única llamada a un modelo de lenguaje. FinRobot propone un marco de agentes para análisis de compañías y valoración, combinando razonamiento cuantitativo y cualitativo mediante agentes especializados [11]. TradingAgents plantea una arquitectura multiagente inspirada en mesas de negociación, con perfiles diferenciados para análisis fundamental, sentimiento, análisis técnico, debate entre posiciones alcistas y bajistas, gestión de riesgo y toma de decisiones de compraventa [12].

Estos sistemas son cercanos al presente trabajo porque dividen tareas complejas en componentes especializados. Sin embargo, su objetivo es más ambicioso y también más arriesgado desde el punto de vista financiero: aproximarse a informes de inversión, valoración o decisiones de compraventa. Este TFM adopta una posición más conservadora. El sistema no genera tesis de inversión, no recomienda comprar o vender activos y no incorpora juicio discrecional sobre una compañía. Su alcance se limita al análisis histórico-descriptivo de datos ya descargados.

La comparación permite delimitar mejor la contribución del proyecto. Frente a agentes financieros orientados a asesoramiento o compraventa, aquí se priorizan reproducibilidad, trazabilidad y control del flujo: entrada estructurada, selección de intención, cálculo determinista, ejecución controlada y redacción final asistida.

2.4. Agentes con herramientas, planificación y ejecución

En el plano técnico, el diseño del sistema se relaciona con la literatura sobre agentes basados en modelos de lenguaje. ReAct introdujo la combinación de razonamiento y actuación, permitiendo que un modelo alterne entre pasos de razonamiento y acciones sobre herramientas externas [13]. Toolformer mostró que los modelos pueden aprender

a decidir cuándo llamar a herramientas externas, como calculadoras, buscadores o sistemas de preguntas y respuestas [14]. Las revisiones sobre agentes autónomos basados en LLM describen esta tendencia como un paso desde modelos conversacionales hacia sistemas capaces de planificar, actuar y observar resultados en entornos externos [15].

Una línea reciente dentro de los LLM son los denominados modelos de razonamiento, diseñados para dedicar más cómputo de inferencia a descomponer problemas, generar pasos intermedios y revisar la respuesta antes de emitir la salida final. La familia OpenAI o1 popularizó este enfoque en tareas complejas de ciencia, programación y matemáticas [16]; DeepSeek-R1 mostró que el razonamiento puede incentivarse mediante aprendizaje por refuerzo y destilarse posteriormente a modelos más pequeños [17]; y la familia `gpt-oss` se presenta como un conjunto de modelos abiertos de razonamiento con capacidades de uso de herramientas y trazas intermedias [7]. Esta línea es relevante para el TFM porque el modelo `openai/gpt-oss-20b` pertenece a dicha familia, aunque el trabajo no evalúa el razonamiento como capacidad principal ni delega en él el cálculo financiero.

El TFM se inspira en esta idea general de separación entre razonamiento y uso de herramientas, pero no delega todo el control en el modelo. La arquitectura fija de antemano los nodos del flujo de trabajo: ingesta, enrutado, planificación, generación de código, ejecución e interpretación. Esta decisión reduce flexibilidad frente a un agente autónomo, pero aumenta control, depuración y reproducibilidad. En un dominio como el financiero, esta restricción es deseable porque evita que el modelo improvise cálculos, columnas o fuentes de datos.

Desde esta perspectiva, LangGraph se utiliza como referencia práctica para construir flujos de trabajo con estado compartido y nodos diferenciados [18]. No obstante, la contribución del TFM no depende de una herramienta concreta, sino del patrón de diseño: dividir una tarea analítica en etapas verificables y limitar la intervención generativa a las fases donde aporta valor sin comprometer el cálculo.

2.5. Generación de código y análisis de datos

La generación automática de código es una de las capacidades más relevantes de los LLM. El trabajo sobre Codex y HumanEval mostró que los modelos entrenados con código pueden sintetizar programas a partir de descripciones en lenguaje natural, aunque la evaluación debe centrarse en la corrección funcional y no solo en la apariencia del código [19]. En ciencia de datos, DS-1000 propuso un banco de prueba específico para generación de código en tareas realistas con librerías habituales de análisis de datos [20].

Estos trabajos justifican que la generación de código sea una línea natural para sistemas de análisis financiero asistidos por modelos de lenguaje. Sin embargo, también

muestran un riesgo importante: el código generado debe ejecutarse, validarse y compararse con el resultado esperado. En el presente TFM, la generación libre mediante LLM se deja como extensión futura. La evaluación principal utiliza plantillas deterministas y programas controlados para evitar errores silenciosos en métricas financieras.

La decisión es deliberada. En un prototipo académico cuyo objetivo es demostrar arquitectura y trazabilidad, resulta más valioso garantizar que el cálculo de rentabilidades, volatilidad, caída máxima o indicadores técnicos procede de código auditable que maximizar la autonomía generativa del sistema desde la primera versión.

2.6. Fiabilidad, alucinaciones y trazabilidad

Una limitación conocida de los modelos generativos es la posibilidad de producir respuestas plausibles pero no fieles a los datos. Las revisiones sobre alucinaciones en generación de lenguaje natural describen este fenómeno como una falta de correspondencia entre la salida generada y la fuente de información o el conocimiento verificable [21]. En tareas de conocimiento, la generación aumentada con recuperación propuso una forma de reducir este problema mediante el apoyo explícito en fuentes externas recuperadas durante la generación [22].

Aunque este TFM no implementa un sistema RAG documental, adopta una lógica de anclaje similar: el modelo no inventa datos financieros, sino que recibe resultados calculados previamente por el flujo de análisis. La fuente de verdad son los CSV descargados mediante `yfinance` y los cálculos programáticos realizados sobre ellos [23], [24]. De esta forma, la capa LLM se utiliza para mejorar la explicación, no para decidir las cifras.

Este planteamiento también responde a una exigencia metodológica. En análisis financiero, una respuesta natural pero no trazable tiene poco valor académico. El sistema debe poder explicar qué datos se han usado, qué cálculo se ha realizado y qué limitaciones tiene la interpretación. Por ello, la memoria insiste en separar datos, cálculo, ejecución e interpretación.

2.7. Comparativa con trabajos relacionados

La Tabla 2.1 resume la relación entre trabajos existentes y la propuesta desarrollada. La comparación se centra en el objetivo, el tipo de solución y la diferencia principal con este TFM.

Trabajo	Dominio	Enfoque	Relación con este TFM
FinBERT [8]	PLN financiero	Modelo especializado para sentimiento financiero.	Muestra la utilidad de adaptar lenguaje al dominio financiero, pero no aborda flujos multiagente ni cálculo histórico trazable.
BloombergGPT [9]	LLM financiero	Modelo grande entrenado con datos financieros y generales.	Se centra en entrenamiento y evaluación de un modelo financiero; este TFM integra modelos existentes sin entrenarlos.
FinGPT [10]	LLM financiero abierto	Recursos abiertos y flujo de datos para modelos financieros.	Comparte la preocupación por datos y apertura, pero el TFM se enfoca en una arquitectura de análisis histórico con cálculo determinista.
FinRobot [11]	Agentes financieros	Sistema multiagente para análisis de compañías y valoración.	Es más ambicioso en análisis fundamental; este TFM evita valoración y recomendación de inversión.
TradingAgents [12]	Compraventa multiagente	Agentes especializados para debate, riesgo y decisión de mercado.	Usa agentes para decisiones de mercado; este TFM se limita a análisis descriptivo y no produce señales de compra o venta.
ReAct y Toolformer [13], [14]	Agentes con herramientas	Razonamiento combinado con acciones y llamadas a herramientas.	Inspiran la separación entre razonamiento y herramientas, pero aquí el flujo está restringido y validado de forma programática.
OpenAI o1, DeepSeek-R1 y gpt-oss [7], [16], [17]	Modelos de razonamiento	Uso de pasos intermedios, aprendizaje por refuerzo o mayor cómputo de inferencia para resolver tareas complejas.	Contextualizan el campo reasoning observado en el servidor vLLM, pero el TFM no usa esas trazas para calcular métricas financieras.
Codex y DS-1000 [19], [20]	Generación de código	Síntesis y evaluación de código funcional o de ciencia de datos.	Justifican la línea futura de código generado, aunque el MVP usa plantillas deterministas para preservar trazabilidad.

Tabla 2.1: Comparativa entre trabajos relacionados y la propuesta del TFM.

2.8. Posicionamiento de la propuesta

La revisión muestra que existen tres familias de trabajos cercanos: modelos financieros especializados, agentes financieros orientados a análisis o compraventa, y arquitecturas generales de LLM con herramientas y generación de código. El TFM se sitúa en la intersección de estas líneas, pero adopta una estrategia más acotada.

La aportación principal no es entrenar un nuevo modelo financiero ni construir un asesor de inversión. La propuesta consiste en diseñar y validar un flujo multiagente para análisis financiero histórico en el que el cálculo permanece controlado por código y el LLM se incorpora de forma gradual para mejorar la interpretación textual. Esta posición permite aprovechar la expresividad de los modelos de lenguaje sin perder trazabilidad, reproducibilidad ni control sobre las métricas financieras.

Capítulo 3

Datos y material de trabajo

3.1. Fuente de datos y trazabilidad

El sistema trabaja con datos financieros históricos descargados previamente mediante `yfinance` [23] y almacenados en ficheros CSV. Esta librería permite acceder de forma programática a datos disponibles en Yahoo Finance [24]. Cada fichero de datos dispone de un fichero de metadatos asociado en formato JSON, donde se conserva información sobre la consulta original, la intención analítica, los símbolos financieros implicados, el intervalo temporal y los parámetros utilizados para la descarga.

Esta decisión permite separar la fase de obtención de datos de la fase de análisis. El MVP no depende de que la API externa esté disponible durante cada ejecución, sino que opera sobre un conjunto de datos congelado y versionado. De este modo, los resultados obtenidos son más reproducibles y pueden revisarse posteriormente a partir de los mismos ficheros de entrada.

3.2. Catálogo de consultas, activos y formato de entrada

En la versión actual se han definido diez consultas de referencia y se han analizado ocho CSV, que contienen diez series normalizadas por símbolo financiero y caso de uso. Los activos incluidos cubren acciones, ETFs, un índice amplio, un criptoactivo, una divisa y un futuro de materia prima.

Cada consulta se transforma en una representación estructurada antes de entrar en el flujo multiagente. Esta representación no expone detalles internos del proyecto, sino los elementos semánticos necesarios para que el sistema pueda seleccionar el análisis adecuado y recuperar los datos correspondientes. La Tabla 3.1 resume el formato utilizado.

La variedad de activos permite comprobar que el sistema no está limitado a un

Elemento	Descripción	Ejemplo
Consulta original	Texto introducido o definido como caso de prueba.	Comparar Nvidia y AMD en dos años.
Intención analítica	Tipo de análisis que debe ejecutarse.	Comparación de activos.
Símbolos financieros	Activos sobre los que se realiza el análisis.	NVDA, AMD.
Rango temporal	Periodo relativo o fechas de inicio y fin.	Dos años.
Intervalo de observación	Frecuencia de los datos históricos.	Diario.
Estado de aclaración	Indica si faltan datos para ejecutar la consulta.	No requiere aclaración.
Avisos	Observaciones detectadas durante la preparación de la entrada.	Sin avisos.

Tabla 3.1: Estructura semántica de una consulta financiera antes de su procesamiento.

Categoría	Activos incluidos
Acciones	AAPL, AMD, NVDA
ETFs	QQQ, SPY
Índice	S&P 500 (^GSPC)
Criptoactivo	BTC-USD
Divisa	EURUSD=X
Materia prima / futuro	GC=F

Tabla 3.2: Tipos de activos representados en el conjunto de datos.

único instrumento financiero y que puede trabajar con diferentes horizontes temporales e intervalos de observación.

3.3. Estructura de los CSV

Los ficheros generados por `yfinance` contienen columnas de tipo OHLCV: apertura, máximo, mínimo, cierre, cierre ajustado y volumen. En algunos casos se almacenan con cabeceras multinivel, por lo que el proyecto incorpora una capa de lectura y normalización antes de ejecutar los cálculos financieros.

La variable más importante para el MVP es el precio de cierre, ya que sobre ella se calculan crecimiento, rentabilidades, volatilidad, máxima caída acumulada e indicadores técnicos básicos. El volumen se conserva como información complementaria, aunque no se trata como obligatorio para todos los activos, especialmente en divisas o futuros intradía.

3.4. Análisis exploratorio de datos

El análisis exploratorio confirma que el conjunto de datos contiene 5.080 filas normalizadas en formato largo, con un rango temporal observado entre el 2 de enero de 2020 y el 17 de marzo de 2026. Los intervalos considerados son diario (1d) e intradía horario (1h).

Indicador	Resultado
Casos definidos en catálogo	10
CSV analizados	8
Series por símbolo y caso	10
Filas normalizadas	5.080
Intervalos	1d, 1h
Nulos en precios de cierre	0
Duplicados por fecha	0 en los ficheros analizados

Tabla 3.3: Resumen de cobertura y calidad del conjunto de datos.

Además del resumen agregado, se calcularon métricas descriptivas por serie para comprobar que los casos de prueba cubrían situaciones financieras diversas. La Tabla 3.4 muestra una selección de las series analizadas. Se observan activos con crecimientos muy elevados, como NVDA en cinco años, series con caídas acumuladas significativas, como AMD en la comparativa con NVDA, y activos de menor volatilidad relativa, como SPY. Esta diversidad es útil porque obliga al sistema a redactar respuestas para escenarios con magnitudes y niveles de riesgo distintos.

Serie	Retorno total	Volatilidad anual	Máx. caída
NVDA, 5 años	1273.33 %	51.31 %	-66.36 %
BTC-USD, 2024	109.76 %	44.13 %	-26.18 %
NVDA, 2 años	107.13 %	49.42 %	-36.89 %
S&P 500, desde 2020	105.64 %	20.77 %	-33.93 %
QQQ, 2024	28.07 %	17.99 %	-13.56 %
SPY, 2024	24.45 %	12.63 %	-8.41 %
AMD, 2 años	3.11 %	55.10 %	-58.98 %
AAPL, 3 meses	-7.00 %	24.15 %	-10.07 %

Tabla 3.4: Métricas exploratorias por serie utilizadas para contextualizar los casos de prueba.

3.5. Calidad, cobertura y limitaciones

La ausencia de nulos en el precio de cierre es relevante porque las métricas principales del MVP dependen directamente de esta columna. También se comprobó la

ausencia de duplicados por fecha en los ficheros analizados. Estas condiciones permiten ejecutar las seis intenciones actuales sin introducir imputaciones adicionales.

Como limitación, las descargas basadas en periodos relativos, por ejemplo tres meses o cinco años, dependen de la fecha en la que se generó el CSV. Para una versión final completamente cerrada sería recomendable fijar fechas explícitas de inicio y fin en todos los casos. También debe considerarse que los datos proceden de Yahoo Finance mediante `yfinance`. Al tratarse de una herramienta abierta no afiliada ni avalada por Yahoo, su uso se considera adecuado para un contexto académico y experimental, pero en un entorno productivo habría que estudiar licencias, disponibilidad, estabilidad del proveedor y derechos de uso de los datos descargados [23].

3.6. Papel de los cuadernos auxiliares

Durante el desarrollo se conservaron tres cuadernos auxiliares con funciones diferenciadas. El primero se utilizó como laboratorio para probar nuevos símbolos financieros, periodos e intervalos antes de incorporarlos al conjunto de datos definitivo. El segundo sirvió para documentar la relación entre las consultas de referencia y los CSV esperados. El tercero quedó como referencia principal del análisis exploratorio formal y de las tablas agregadas utilizadas en esta memoria.

Capítulo 4

Metodología

4.1. Enfoque incremental del desarrollo

El desarrollo del trabajo se ha planteado como una sucesión de iteraciones controladas. En lugar de construir desde el inicio un sistema completamente dependiente de modelos de lenguaje, se ha partido de un MVP determinista capaz de completar el flujo de extremo a extremo. Esta decisión permite validar primero la arquitectura, la trazabilidad y la ejecución del código generado antes de introducir componentes generativos más variables.

La primera iteración validó el flujo mínimo con dos intenciones analíticas: crecimiento histórico y comparación de activos. La segunda iteración amplió el alcance con análisis descriptivo y análisis de retornos. La tercera incorporó análisis de riesgo histórico, análisis técnico, pruebas de error y una reorganización interna basada en familias de agentes. La Tabla 4.1 resume la progresión seguida.

Iteración	Objetivo principal	Evidencia obtenida
MVP 1	Validar el flujo extremo a extremo con <code>price_growth</code> y <code>compare_assets</code> .	El sistema procesa un caso de activo único y un caso multi-activo, genera código ejecutable y produce una respuesta interpretable.
MVP 2	Comprobar que la arquitectura crece sin rediseñarse, incorporando <code>asset_overview</code> y <code>return_analysis</code> .	Se pasa de dos a cuatro intenciones y se valida que las nuevas métricas se integran sobre las mismas interfaces.
MVP 3	Aumentar madurez funcional y robustez mediante <code>historical_risk_analysis</code> , <code>technical_analysis</code> y pruebas negativas.	Se consolidan seis intenciones, tres escenarios de error controlado y una modularización por familias de agentes.

Tabla 4.1: Evolución incremental del MVP.

4.2. Definición de intenciones analíticas

El MVP trabaja con un conjunto cerrado de intenciones. Esta restricción hace que el problema sea evaluable y reduce la ambigüedad del sistema. Cada intención se asocia con una familia de agente, un plan analítico, un bloque de código y una forma de interpretar la salida.

Intención	Objetivo
<code>price_growth</code>	Calcular crecimiento o caída de un activo en un periodo.
<code>compare_assets</code>	Comparar la evolución relativa de varios activos.
<code>asset_overview</code>	Generar una visión descriptiva general de un activo.
<code>return_analysis</code>	Analizar retornos históricos y estadísticos básicos.
<code>historical_risk_analysis</code>	Estimar riesgo histórico mediante volatilidad y draw-down.
<code>technical_analysis</code>	Calcular indicadores técnicos como medias móviles y RSI.

Tabla 4.2: Intenciones analíticas soportadas por el MVP.

4.3. Diseño experimental

La evaluación se apoya en seis casos representativos, uno por cada intención implementada. Estos casos permiten comprobar que el sistema valida la entrada, selecciona la familia analítica correcta, genera un plan, construye código ejecutable, ejecuta el script y devuelve una respuesta final interpretable.

Además de los casos correctos, se incorporan escenarios negativos para comprobar la robustez del flujo. Entre ellos se incluyen intenciones no soportadas, rutas CSV inexistentes y cardinalidad incorrecta de tickers para análisis que requieren una única serie.

4.4. Métricas de validación

La validación del MVP no se plantea como entrenamiento de un modelo supervisado, sino como evaluación funcional de un pipeline software. Por tanto, las métricas principales son:

- Éxito de ejecución del flujo completo.
- Código de retorno del script generado.
- Presencia o ausencia de errores controlados.

- Cobertura de intenciones analíticas.
- Uso de fallback determinista cuando falla una API externa.
- Tiempo de respuesta en las pruebas con proveedores LLM.

4.5. Justificación de no entrenar un modelo propio

El objetivo del TFM no es entrenar desde cero un modelo financiero, sino estudiar una arquitectura multiagente trazable para automatizar análisis históricos. Por ello, el trabajo utiliza modelos de lenguaje preentrenados a través de APIs compatibles con OpenAI y limita su papel, en la configuración actual, a la fase de interpretación final.

Esta decisión reduce el coste computacional, evita depender de hardware GPU propio y permite centrar la evaluación en la robustez del workflow. La planificación, el cálculo financiero y la ejecución permanecen bajo control programático.

4.6. Gestión de riesgos técnicos

Los principales riesgos identificados son la fragilidad del código generado, la disponibilidad de APIs externas, los problemas de codificación en respuestas LLM, las cuotas de proveedores gratuitos y la posible confusión entre análisis histórico y recomendación financiera. Para mitigarlos, el sistema conserva una base determinista, valida entradas, captura salidas de ejecución, registra artefactos y activa la capa LLM de forma controlada, empezando por la interpretación final.

Capítulo 5

Diseño del sistema

5.1. Arquitectura general

El sistema se ha diseñado como un workflow modular que transforma una consulta financiera estructurada en una respuesta analítica basada en cálculos ejecutados sobre datos históricos. La arquitectura separa de forma explícita las fases de ingesta, enrutado, planificación, generación de código, ejecución e interpretación.

Esta separación evita que una única llamada generativa concentre todo el razonamiento del sistema. Cada fase tiene una responsabilidad concreta y produce artefactos intermedios que pueden revisarse, probarse y sustituirse de forma independiente.

5.2. Entrada estructurada y estado compartido

El flujo parte de una entrada normalizada que incluye consulta original, intención analítica, símbolos financieros, periodo o fechas, intervalo y datos históricos asociados. A partir de esa entrada se construye un estado compartido que viaja por los nodos del flujo.

El estado contiene, entre otros campos, la intención detectada, el agente seleccionado, el plan analítico, el código generado, la salida estándar, la salida de error, el código de retorno, los avisos y la respuesta final. Esta estructura permite mantener trazabilidad entre la entrada inicial y el resultado entregado al usuario.

5.3. Nodos del flujo y comunicación entre componentes

La Figura 5.1 muestra la estructura del flujo multiagente y las comunicaciones entre nodos. Las flechas continuas representan la ruta principal de ejecución; las flechas rojas recogen las salidas de error controladas; y las flechas discontinuas indican puntos

donde puede intervenir una capa LLM opcional, siempre con fallback determinista. La comunicación entre componentes no se realiza mediante texto libre entre agentes, sino a través de un estado compartido, lo que permite auditar qué información lee y actualiza cada fase.

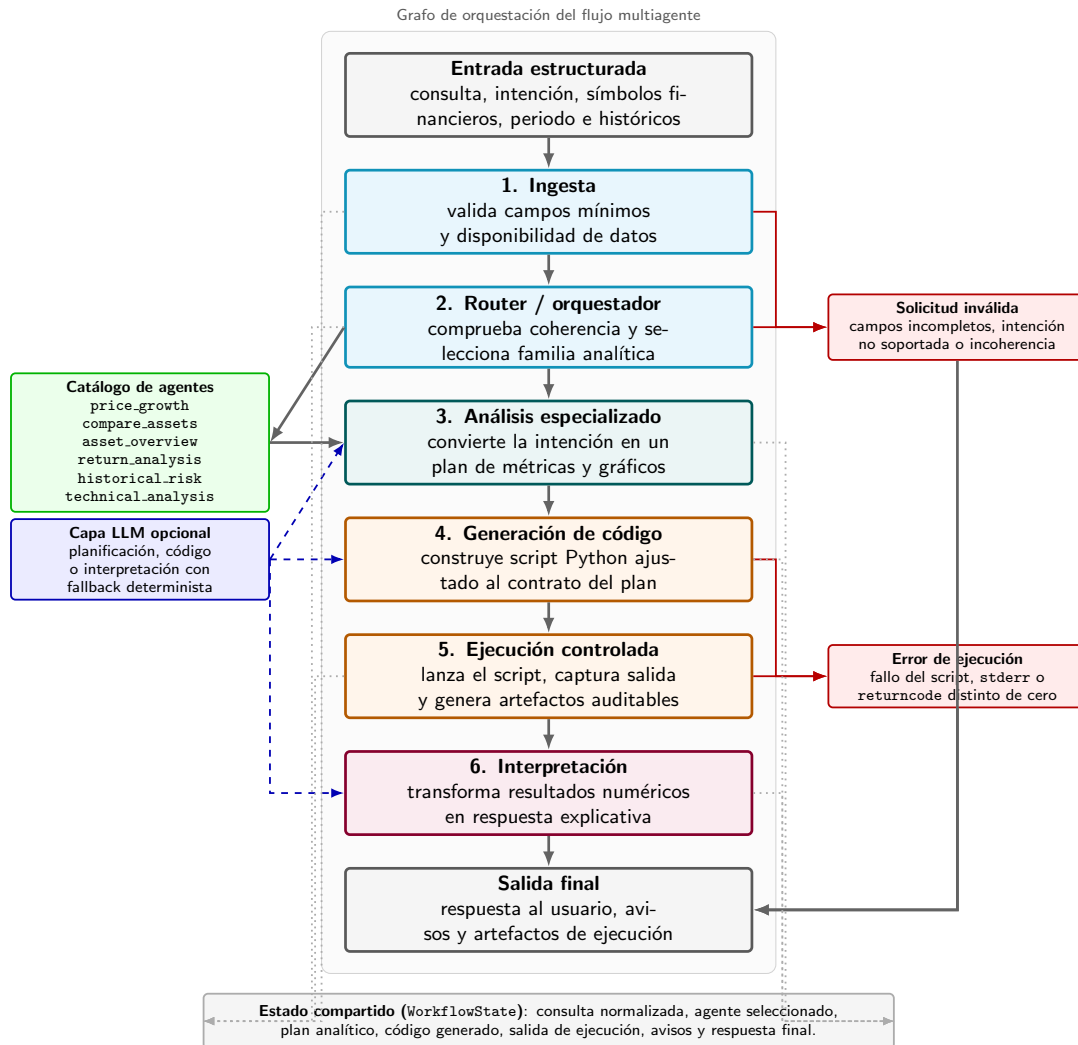


Figura 5.1: Arquitectura de nodos del flujo multiagente y comunicación mediante estado compartido.

El flujo se organiza en seis nodos principales:

1. **Ingesta**: valida la entrada mínima y comprueba la existencia de los ficheros.
2. **Router**: valida la coherencia de la solicitud y selecciona la familia analítica.
3. **Análisis especializado**: transforma la intención en un plan estructurado.
4. **Generación de código**: construye un script Python adaptado al plan.
5. **Ejecución**: ejecuta el script en un proceso controlado.

6. **Interpretación:** convierte los resultados en una respuesta legible.

Esta organización separa las decisiones de control del cálculo financiero. El router no ejecuta análisis, sino que valida la solicitud y selecciona la familia adecuada. El nodo especialista no calcula directamente sobre los datos, sino que produce un plan analítico estructurado. La generación de código transforma ese plan en un script reproducible y la ejecución se realiza fuera del proceso principal, capturando salida estándar, errores y artefactos. Finalmente, el nodo de interpretación redacta la respuesta a partir de resultados ya calculados, evitando que el modelo invente métricas o sustituya el cálculo programático.

5.4. Familias de agentes analíticos

La lógica específica de cada intención se agrupa en familias de agentes. En esta versión, la idea de agente no se interpreta como una entidad autónoma sin restricciones, sino como un componente especializado con un contrato claro. Cada agente sabe qué métricas calcular, qué salidas esperar y cómo construir una interpretación inicial.

Esta organización facilita la extensión del sistema. Si se desea incorporar una nueva intención, se puede añadir una nueva familia analítica sin reescribir el workflow completo.

5.5. Generación controlada de código

La generación de código se sitúa entre el plan analítico y la ejecución. En el modo determinista, el sistema utiliza plantillas programáticas que producen scripts Python ajustados al contrato esperado. Esta decisión permite controlar imports, rutas, formato de salida y gestión de errores.

En futuras iteraciones, la generación de código podría delegarse parcialmente en un modelo de lenguaje. Sin embargo, esta activación se considera más arriesgada que la interpretación final, por lo que el MVP actual mantiene esta fase bajo control determinista.

5.6. Ejecución segura y artefactos

El código se guarda como un script independiente y se ejecuta mediante un proceso externo controlado. La ejecución captura `stdout`, `stderr`, `returncode` y un payload estructurado con los resultados del análisis.

Los scripts generados y los logs de ejecución se almacenan como artefactos revisables. Esto permite auditar qué cálculos se realizaron y qué salida se utilizó para construir la respuesta final.

5.7. Gestión de errores y fallback determinista

El sistema incorpora validaciones para errores previsibles, como intenciones no soportadas, CSV inexistentes o número incorrecto de tickers. Cuando se activa la capa LLM, el flujo conserva un fallback determinista. Si la API falla, no está configurada o devuelve una respuesta no válida, el sistema puede continuar con la lógica programática.

Esta estrategia es clave para que el MVP no dependa críticamente de proveedores externos ni de disponibilidad de red.

Capítulo 6

Implementación

6.1. Estructura del repositorio

El código del proyecto se organiza en una estructura separada por responsabilidades. La carpeta `src/` contiene la implementación principal, `data/` conserva los datos y catálogos, `notebooks/` incluye el análisis exploratorio y `tests/` agrupa las pruebas automatizadas.

Ruta	Función
<code>src/schemas.py</code>	Define modelos de entrada, salida y estado.
<code>src/graph/</code>	Contiene nodos, routing y construcción del workflow.
<code>src/agents/</code>	Agrupar las familias analíticas del MVP.
<code>src/execution/</code>	Gestiona persistencia y ejecución de scripts.
<code>src/io/</code>	Lee y normaliza los CSV financieros.
<code>src/llm/</code>	Prepara la integración controlada con APIs LLM.
<code>tests/</code>	Valida casos funcionales y escenarios de error.

Tabla 6.1: Principales módulos del repositorio.

6.2. Módulos principales

El fichero `schemas.py` define los contratos de datos que atraviesan el sistema. Los módulos de `src/graph/` implementan los nodos del pipeline y las reglas de transición. Las familias de agentes se concentran en `src/agents/`, donde cada intención se asocia con una lógica analítica concreta.

La ejecución de código se encapsula en `src/execution/code_runner.py`. Este módulo guarda el script generado, lo lanza como proceso externo y devuelve salidas estructuradas. Esta separación evita mezclar el cálculo financiero con la orquestación del workflow.

6.3. Ejecución, despliegue local y reproducibilidad

El MVP se ejecuta de forma local mediante `run_mvp.py`. La memoria considera este despliegue local como suficiente para la fase académica del trabajo, ya que el objetivo principal es validar el pipeline y no publicar un servicio productivo.

```
python run_mvp.py --example growth_nvda
python run_mvp.py --example compare_nvda_amd
python run_mvp.py --example overview_aapl
python run_mvp.py --example returns_qqq_spy
python run_mvp.py --example risk_qqq_spy
python run_mvp.py --example technical_aapl
```

La reproducibilidad se apoya en tres elementos: datos CSV versionados, ejemplos de entrada controlados y pruebas automatizadas. Las claves de API se almacenan en un fichero local `.env`, excluido del control de versiones.

6.4. Tests automatizados

El proyecto incorpora pruebas unitarias para validar tanto el modo determinista como la configuración LLM. La batería principal comprueba las seis intenciones soportadas y varios errores previsibles.

```
python -m unittest tests/test_mvp.py
python -m unittest tests/test_llm_config.py tests/test_mvp.py
```

En las ejecuciones documentadas, el MVP supera nueve pruebas sobre nueve en la batería funcional principal. Al incluir las pruebas de configuración LLM, se documenta una ejecución de catorce pruebas superadas, lo que confirma que la incorporación de dependencias LLM no rompe el funcionamiento determinista.

6.5. Evaluación automática de perfiles LLM

Para comparar el modo determinista con los distintos proveedores LLM se incorporó el script `scripts/evaluate_llm_profiles.py`. Este script ejecuta una misma batería de ejemplos contra varios perfiles: `deterministic`, `groq`, `gemini` y `university`. Su objetivo es evitar evaluaciones manuales aisladas y generar evidencias comparables entre configuraciones.

Cada ejecución registra el perfil utilizado, el ejemplo, la intención analítica, el estado final, el código de retorno del script generado, posibles avisos, la respuesta final, el agente seleccionado, el plan analítico, la salida estructurada de ejecución y el

tiempo aproximado. También se incorporan comprobaciones cualitativas simples, como detectar si se ha usado fallback o si la respuesta evita formulaciones propias de una recomendación directa de inversión.

```
python scripts/evaluate_llm_profiles.py \  
    --profiles deterministic groq gemini university --examples all
```

Los resultados generados se consideran artefactos de evaluación y no código fuente. Por ello, las salidas completas pueden almacenarse en una carpeta de resultados ignorada por el control de versiones, mientras que la memoria recoge los indicadores agregados más relevantes.

6.6. Artefactos generados

Cada ejecución puede producir scripts, logs y payloads de resultado. Estos artefactos permiten revisar el comportamiento del sistema después de la ejecución y aportan evidencia para la memoria. La existencia de scripts generados también facilita detectar errores en la fase de cálculo, en lugar de confiar únicamente en la respuesta final.

Capítulo 7

Integración controlada de modelos de lenguaje

7.1. Motivación de la capa LLM

Una vez consolidado el MVP determinista, el trabajo incorpora una capa de modelos de lenguaje. El objetivo no es sustituir el cálculo financiero, sino mejorar la redacción final y estudiar la portabilidad del sistema entre distintos proveedores.

La configuración recomendada activa inicialmente el LLM solo para interpretación. De este modo, el modelo recibe resultados ya calculados y redacta una explicación más natural, pero no modifica la selección del agente, el plan analítico ni el código ejecutado.

7.2. Cliente compatible con OpenAI

La integración se ha preparado mediante un cliente compatible con APIs tipo OpenAI. Esta decisión permite alternar entre proveedores sin modificar el workflow principal. El sistema utiliza variables de entorno para seleccionar perfil, modelo, URL base y clave de API.

En esta memoria se denomina **servidor vLLM** a la configuración desplegada en el entorno universitario que expone una API compatible con OpenAI Chat Completions mediante vLLM. Esta denominación se utiliza únicamente para identificar la configuración de ejecución; el modelo servido en las pruebas es `openai/gpt-oss-20b` [7]. Por tanto, vLLM no se presenta como proveedor ni como modelo, sino como la infraestructura que permite servir el modelo con una interfaz compatible.

7.3. Activación granular

La capa LLM puede activarse por fases mediante variables de configuración:

- `LLM_USE_FOR_INTERPRETATION`: el modelo redacta la respuesta final.
- `LLM_USE_FOR_PLANNING`: el modelo puede ajustar el plan analítico.
- `LLM_USE_FOR_CODEGEN`: el modelo puede generar código Python.

La activación de planificación y generación de código se mantiene deshabilitada en la evaluación principal, porque son fases con mayor riesgo de introducir errores no trazables.

7.4. Configuraciones evaluadas

Se han preparado y evaluado tres configuraciones principales:

- **Groq**: proveedor externo compatible con OpenAI, utilizado como primera API operativa con el modelo `llama-3.3-70b-versatile`. Este identificador corresponde al despliegue en Groq de Meta Llama 3.3 70B Instruct [2], [3], [4].
- **Gemini**: proveedor secundario para estudiar portabilidad y calidad comparativa, usando el modelo `gemini-2.5-flash` [5], [6].
- **Servidor vLLM**: servidor compatible con OpenAI Chat Completions habilitado en el entorno universitario, usando el modelo `openai/gpt-oss-20b` [7].

7.5. Estrategia de fallback

El diseño conserva siempre el modo determinista como fallback. Si una API falla por red, cuota, indisponibilidad o error de formato, el sistema puede devolver una respuesta programática válida. Esta propiedad es esencial para evitar que el MVP quede bloqueado por dependencias externas.

Durante las pruebas se observaron errores de cuota y disponibilidad en Gemini, así como un problema inicial de red en el entorno de ejecución. También se comprobó que, si falta la dependencia `openai`, el sistema no queda inutilizado: registra el aviso correspondiente y mantiene una respuesta determinista. En todos estos casos, el enfoque con fallback permitió mantener el sistema operativo.

7.6. Observaciones técnicas de la integración

La primera integración real con APIs externas puso de manifiesto varias cuestiones prácticas. Groq completó la batería de seis casos con `llama-3.3-70b-versatile` sin warnings una vez instaladas las dependencias y habilitada la conectividad, por lo que se consideró el proveedor externo más cómodo para iterar inicialmente. Gemini también produjo respuestas alineadas con los datos mediante `gemini-2.5-flash`, pero mostró limitaciones operativas del *free tier*: errores 429 por cuota y 503 por alta demanda temporal, resueltos mediante espera, reintento y fallback determinista.

El servidor vLLM resultó especialmente relevante para el TFM porque confirma que la arquitectura puede conectarse a un modelo servido localmente mediante una API compatible con OpenAI Chat Completions sin cambiar el workflow principal. En la validación técnica se utilizó el modelo `openai/gpt-oss-20b` [7], que pertenece a la familia de modelos abiertos de razonamiento descrita en el estado del arte. Al servirse mediante vLLM, el servidor puede devolver un campo adicional `reasoning`, separado del campo `content`, con información interna o intermedia asociada a la generación de la respuesta. En este TFM, dicho campo se conserva únicamente en la respuesta cruda con fines de depuración y auditoría, pero no se utiliza para calcular métricas, seleccionar agentes ni construir la respuesta final al usuario. La aplicación emplea exclusivamente el contenido final del mensaje, manteniendo el cálculo financiero dentro del flujo programático. También se incorporó una reparación defensiva de codificación para evitar mojibake en respuestas con tildes o símbolos especiales.

7.7. Gestión de credenciales

Las claves de Groq, Gemini y del servidor vLLM se gestionan mediante variables de entorno en el fichero local `.env`. Este archivo no se versiona y no debe compartirse. El repositorio incluye configuraciones de ejemplo, pero una ejecución real con API solo es posible cuando el entorno local contiene una clave válida. Si la clave falta, es un marcador de ejemplo o el perfil no está configurado, el sistema debe continuar en modo determinista o activar fallback.

Capítulo 8

Resultados y evaluación

Este capítulo recoge la evaluación del prototipo desarrollado. A diferencia de una validación basada únicamente en ejemplos correctos, se han utilizado tres niveles de prueba: casos funcionales, escenarios de error controlado y comparación entre el modo determinista y varias configuraciones con modelos de lenguaje. El objetivo no es demostrar que el sistema prediga el mercado, sino comprobar si la arquitectura ejecuta análisis históricos de forma trazable, si falla de manera controlada y si el uso de LLM aporta valor frente a una salida puramente programática.

8.1. Protocolo de evaluación

La evaluación se ha diseñado alrededor de las seis intenciones analíticas soportadas por el MVP: crecimiento de precio, comparación de activos, visión descriptiva de un activo, análisis de retornos, análisis de riesgo histórico y análisis técnico. Para cada ejecución se registran el estado final del flujo, el agente seleccionado, el código de retorno del proceso de análisis, el tiempo de ejecución, los avisos generados, la respuesta final y una comprobación básica de seguridad para detectar recomendaciones directas de inversión.

Se han considerado dos modos principales. El primero es el modo determinista, donde la selección de agente, el plan de análisis, el código ejecutado y la respuesta textual proceden de reglas y plantillas del propio sistema. En esta memoria se utiliza como *baseline*: una referencia inicial reproducible contra la que comparar las variantes con LLM. Esta decisión responde a la estrategia seguida al comienzo del proyecto: antes de incorporar modelos generativos, se necesitaba una versión controlada que permitiera comprobar que el flujo funcionaba, que las métricas financieras eran correctas y que los errores podían gestionarse sin depender de servicios externos.

El segundo modo mantiene el cálculo financiero igual de determinista, pero activa un LLM únicamente en la fase de interpretación final. Por tanto, los modelos de lenguaje no calculan precios, retornos, volatilidades ni señales técnicas; reciben una salida

estructurada ya calculada y la convierten en una explicación más natural. La comparación entre ambos modos permite distinguir qué mejora realmente al usar LLMs: no la precisión numérica, sino la claridad, la naturalidad y la capacidad de relacionar varias métricas en una respuesta más útil para el usuario.

Las métricas utilizadas son las siguientes:

- **Tasa de comportamiento esperado:** proporción de casos cuyo estado final coincide con el esperado.
- **Casos funcionales completados:** número de consultas válidas que terminan con estado `completed`.
- **Errores controlados:** número de consultas inválidas que terminan en un estado de error esperado y con una explicación para el usuario.
- **Warnings:** avisos producidos durante la ejecución.
- **Violaciones de recomendación de inversión:** presencia de expresiones como comprar, vender o recomendar invertir en la respuesta final.
- **Tiempo medio:** duración media observada por grupo de casos.

8.2. Resultados cuantitativos del modo determinista

Para reforzar la evaluación se amplió la batería inicial de seis ejemplos hasta 24 casos. La mitad corresponde a consultas funcionales y la otra mitad a casos de estrés diseñados para forzar errores de entrada, cardinalidades incorrectas, ficheros inexistentes, tickers no presentes en los datos y series temporales insuficientes. La Tabla 8.1 resume los resultados agregados.

Batería	Casos	Resultado esperado	Cumplidos	Avisos	Tiempo medio
Funcional ampliada	12	<code>completed</code>	12/12	0	1.39 s
Estrés y errores	12	Error controlado	12/12	0	0.46 s
Total	24	Comportamiento esperado	24/24	0	0.92 s

Tabla 8.1: Evaluación cuantitativa del flujo determinista ampliado.

Además, no se detectaron recomendaciones directas de inversión en ninguna de las 24 respuestas generadas. Este resultado es relevante porque el sistema está orientado a análisis histórico y no a asesoramiento financiero. La comprobación automática no sustituye a una auditoría lingüística completa, pero permite detectar de forma sistemática expresiones claramente problemáticas.

Los 12 casos funcionales incluyen los seis ejemplos base del MVP y seis variantes adicionales con otros activos y horizontes temporales: Bitcoin, S&P 500, EUR/USD y oro. Esto permite comprobar que el sistema no se limita a una única familia de activos ni a un único formato temporal. Los 12 casos de estrés cubren consultas vacías, intenciones no soportadas, ausencia de tickers, ausencia de CSV, ficheros inexistentes, cardinalidades incompatibles con la intención, tickers que no aparecen en el fichero y series demasiado cortas para calcular retornos o indicadores técnicos.

8.3. Evaluación por familias de agentes

La Tabla 8.2 resume el papel de cada familia de agente. En el estado actual del MVP, los agentes no son LLM autónomos que razonen libremente, sino componentes especializados que construyen un plan de análisis, seleccionan métricas y generan una respuesta determinista o una entrada controlada para la capa de interpretación con LLM.

Agente	Intención	Métricas principales	Flujo y contrato de prompt
<code>growth_agent</code>	<code>price_growth</code>	Precio inicial, precio final, crecimiento absoluto y porcentual	Valida un único ticker y una serie con cierre. El foco textual pide explicar el crecimiento absoluto y porcentual del activo en el rango disponible.
<code>compare_agent</code>	<code>compare_assets</code>	Crecimiento por activo, rendimiento normalizado y ganador por crecimiento	Valida al menos dos tickers. El flujo compara cada activo sobre el periodo común y la respuesta debe destacar el comportamiento relativo sin convertirlo en recomendación.
<code>overview_agent</code>	<code>asset_overview</code>	Primer cierre, último cierre, cambio porcentual, mínimo, máximo y media	Valida un único ticker y resume el comportamiento general. El foco textual prioriza una descripción compacta de niveles representativos del precio.
<code>returns_agent</code>	<code>return_analysis</code>	Retorno acumulado, media diaria, volatilidad, mejor día y peor día	Calcula retornos diarios y compara activos. El contrato textual obliga a explicar retorno y variabilidad observada con datos históricos.
<code>risk_agent</code>	Riesgo histórico	Volatilidad, drawdown máximo, peor día y mejor día	Evalúa riesgo histórico observado. La salida debe hablar de riesgo empírico del periodo, no de riesgo futuro ni de idoneidad de inversión.
<code>technical_agent</code>	Análisis técnico	Último cierre, medias móviles, RSI y señal técnica	Calcula indicadores técnicos sobre un único activo. La respuesta interpreta la señal calculada como lectura histórica, sin traducirla en orden de compra o venta.

Tabla 8.2: Resumen de agentes, métricas y flujo de interpretación.

Cuando se activa un proveedor LLM, el prompt de interpretación recibe dos elementos: la salida estructurada de ejecución y una respuesta determinista de respaldo. La instrucción exige redactar en español, usar las métricas recibidas, no añadir datos externos, no hacer predicciones y evitar recomendaciones de inversión. Esta separación es importante: el LLM mejora la forma del texto final, pero no altera el cálculo ni decide qué agente se ejecuta en las pruebas principales.

8.4. Comparación entre enfoque determinista y uso de LLM

La Tabla 8.3 compara el modo determinista con tres perfiles LLM: Groq con el modelo `llama-3.3-70b-versatile` [2], [3], [4], Gemini con `gemini-2.5-flash` [5], [6] y el servidor vLLM con `openai/gpt-oss-20b` [7]. La comparación se realizó sobre los seis casos representativos iniciales, manteniendo desactivadas la planificación y la generación de código mediante LLM.

La conclusión principal es que los LLM no mejoran la precisión numérica, porque las métricas proceden del mismo pipeline determinista. Su aportación se concentra en la comunicación: hacen que la respuesta sea más cercana a un informe breve, conectan mejor varias métricas en una explicación y reducen la rigidez de las plantillas. A cambio, introducen latencia, dependencia de proveedor y posibles problemas de cuota o disponibilidad.

Un ejemplo representativo se observa en la consulta de crecimiento histórico de NVDA. La respuesta determinista indica que el activo pasa de 13.34 a 183.22 entre 2021-03-17 y 2026-03-16, con una variación del 1273.33 %. El perfil del servidor vLLM expresa los mismos valores con una redacción más narrativa, explicando el aumento absoluto y porcentual en lenguaje natural. El dato no cambia; cambia la capacidad de comunicación del sistema.

8.5. Análisis cualitativo de las propuestas

Además de medir si cada ejecución termina correctamente, se revisó qué tipo de análisis produce cada propuesta. Esta revisión se hizo con una rúbrica cualitativa sencilla: fidelidad a las métricas calculadas, completitud de la explicación, claridad para un usuario no técnico, ausencia de datos externos inventados y ausencia de recomendaciones de inversión. La Tabla 8.4 resume la lectura obtenida.

La Tabla 8.5 muestra la lectura por tipo de consulta. Este análisis es importante porque el valor del LLM no aparece igual en todos los agentes. En consultas simples de crecimiento, la plantilla determinista ya es suficiente; en consultas con varias métricas, la capa LLM resulta más útil porque puede ordenar la explicación y relacionar retorno, volatilidad, drawdown o señales técnicas.

De esta revisión se desprende que el modo determinista es suficiente para validar el cálculo, pero no siempre para comunicar bien el resultado. La capa LLM tiene más sentido cuando la respuesta debe integrar varias métricas y presentarlas de forma comprensible. Por eso, la propuesta final no sustituye el pipeline determinista por un modelo generativo, sino que lo utiliza como capa de interpretación controlada sobre resultados ya calculados.

Configuración	Casos OK	Rango temporal	Lectura del resultado
Determinista	6/6	1.9–3.4 s	Opción más rápida, reproducible y sin dependencias externas. La respuesta es correcta, aunque más rígida.
Groq / llama-3.3-70b-versatile	6/6	3.5–8.1 s	Respuestas naturales y concisas, con baja latencia relativa entre los perfiles LLM evaluados.
Gemini / gemini-2.5-flash	6/6	6.4–10.0 s	Buena calidad textual, pero con mayor sensibilidad operativa a cuotas y disponibilidad del proveedor.
Servidor vLLM / openai/gpt-oss-20b	6/6	4.71–9.87 s	Valida un backend universitario compatible con OpenAI. Completa los seis casos sin fallback ni warnings.

Tabla 8.3: Comparación entre modo determinista y perfiles LLM en la fase de interpretación.

Propuesta	Fortalezas observadas	Debilidades o riesgos
Determinista	Máxima trazabilidad, baja latencia y correspondencia directa entre métrica calculada y frase generada. Es la mejor base para depurar y auditar el sistema.	La respuesta tiende a ser rígida y poco explicativa cuando hay varias métricas relacionadas, por ejemplo en riesgo histórico o análisis técnico.
Groq / llama-3.3-70b-versatile	Buen equilibrio entre naturalidad, concisión y tiempo de respuesta. Las explicaciones conectan mejor las métricas sin alterar los valores calculados.	Depende de un proveedor externo y, como todo LLM, requiere mantener restricciones explícitas para evitar extrapolaciones no deseadas.
Gemini / gemini-2.5-flash	Redacción clara y útil para convertir salidas numéricas en explicaciones más comprensibles. Aporta una segunda referencia para comprobar portabilidad entre proveedores.	Presentó mayor sensibilidad operativa a cuotas y disponibilidad. Esto lo hace menos cómodo como perfil principal de evaluación repetida.
Servidor vLLM / openai/gpt-oss-20b	Demuestra que el sistema puede utilizar un backend universitario compatible con OpenAI sin modificar el flujo. La respuesta final es más narrativa que la plantilla determinista y mantiene las métricas.	La latencia es mayor que en el modo determinista y su calidad depende del modelo desplegado y de la estabilidad del servidor.

Tabla 8.4: Análisis cualitativo de las propuestas evaluadas.

Consulta	Salida determinista	Aportación observada del LLM
Crecimiento histórico	Resume correctamente precio inicial, precio final y variación porcentual.	Mejora la legibilidad, pero aporta poco valor analítico adicional porque la consulta es directa.
Comparación de activos	Identifica el activo con mayor crecimiento y muestra porcentajes.	Explica mejor la diferencia relativa entre activos y evita que la respuesta parezca solo una enumeración de cifras.
Visión descriptiva	Incluye cierre inicial, cierre final, mínimo, máximo y media.	Convierte el conjunto de estadísticos en una descripción más natural del comportamiento del activo.
Retornos	Calcula retorno acumulado, media diaria y volatilidad.	Ayuda a interpretar conjuntamente retorno y variabilidad, que es difícil de comunicar con una frase rígida.
Riesgo histórico	Presenta volatilidad, drawdown y extremos diarios.	Es donde más se aprecia la mejora, porque relaciona varias señales de riesgo sin convertirlas en recomendación.
Análisis técnico	Muestra medias móviles, RSI y señal calculada.	Ordena la lectura técnica y explica la señal como observación histórica, no como instrucción de inversión.

Tabla 8.5: Análisis cualitativo por tipo de consulta evaluada.

8.6. Análisis de robustez y límites

La Tabla 8.6 agrupa los escenarios de estrés utilizados. En todos ellos el comportamiento esperado era que el sistema no continuase como si la consulta fuera válida, sino que devolviera un error controlado. El resultado fue correcto en los 12 casos.

Grupo	Casos evaluados	Comportamiento observado
Entrada inválida	Consulta vacía, intención no soportada, ausencia de ticker, ausencia de CSV y fichero inexistente	El flujo detiene la ejecución y devuelve un estado de error explicable.
Cardinalidad incorrecta	Crecimiento con varios tickers, comparación con un solo ticker, visión general con varios tickers y análisis técnico con varios tickers	El router y la validación impiden seleccionar un flujo incompatible con la consulta.
Datos no alineados	Ticker solicitado que no aparece en el CSV	La ejecución no inventa datos y termina con error controlado.
Datos insuficientes	Retornos e indicadores técnicos con una única fila disponible	El sistema detecta que la serie no permite calcular las métricas solicitadas.

Tabla 8.6: Familias de casos de estrés utilizados para intentar romper el MVP.

Estos resultados muestran que el MVP no solo completa los casos ideales, sino que incorpora barreras para evitar salidas aparentemente válidas sobre entradas inválidas. Sin embargo, la batería sigue siendo limitada respecto a un entorno de producción. No se han evaluado miles de activos, datos en tiempo real, consultas ambiguas generadas por usuarios finales ni degradaciones prolongadas de proveedores externos.

8.7. Por qué no basta con una plantilla de código

Una plantilla de código sería suficiente si el objetivo fuese resolver una única consulta fija, con un único formato de datos y una única salida textual. De hecho, el propio sistema utiliza plantillas deterministas como una pieza interna porque aportan trazabilidad y reproducibilidad. La contribución del trabajo no consiste en negar el valor de las plantillas, sino en integrarlas dentro de una arquitectura capaz de decidir cuándo se pueden aplicar y cómo responder cuando no se pueden aplicar.

Frente a una plantilla aislada, el sistema desarrollado aporta cinco elementos. Primero, normaliza la consulta en una estructura común que separa intención, tickers, rango temporal, intervalo y fuentes de datos. Segundo, valida si la intención y la cardinalidad de activos son compatibles antes de ejecutar código. Tercero, selecciona una familia de agente y un plan analítico explícito, por lo que cada resultado puede trazarse

hasta métricas concretas. Cuarto, ejecuta el análisis en un proceso controlado y captura errores de forma explicable. Quinto, permite activar una capa LLM intercambiable para mejorar la interpretación sin cederle el control del cálculo financiero.

Por tanto, el valor del enfoque multiagente híbrido aparece cuando hay varias consultas posibles, varias familias de análisis, distintos proveedores de interpretación y necesidad de fallback. En ese contexto, una plantilla individual resuelve el cálculo local de un caso, pero no resuelve la orquestación, la validación, la trazabilidad ni la degradación controlada del sistema completo.

8.8. Limitaciones de la evaluación

La evaluación realizada mejora la primera versión de resultados, pero aún presenta limitaciones. El número de casos sigue siendo reducido para afirmar robustez estadística. Las respuestas LLM se han valorado mediante inspección cualitativa y métricas operativas, no mediante un estudio con usuarios ni una rúbrica humana formal. Además, la planificación y la generación de código con LLM permanecen desactivadas en la evaluación principal, precisamente para reducir riesgo y mantener el cálculo bajo control determinista.

También debe tenerse en cuenta que los tiempos de los proveedores LLM dependen de condiciones externas como carga, cuota, red y disponibilidad del endpoint. Por ello, las cifras deben leerse como evidencia experimental del prototipo y no como una comparación universal de rendimiento entre modelos.

8.9. Síntesis

Los resultados respaldan la hipótesis de diseño del trabajo: una arquitectura híbrida permite combinar cálculo financiero determinista con una capa opcional de interpretación mediante LLM. El modo determinista ofrece rapidez, reproducibilidad y control; los perfiles LLM mejoran la claridad comunicativa; y la batería de estrés muestra que el sistema puede fallar de manera controlada ante entradas inválidas. La aportación principal frente a una plantilla de código es la orquestación trazable de varios agentes, métricas y proveedores bajo un flujo común.

Capítulo 9

Aspectos éticos, seguridad y uso responsable

9.1. Delimitación financiera del sistema

El sistema se orienta exclusivamente al análisis financiero histórico y descriptivo. Sus respuestas deben interpretarse como explicaciones basadas en datos pasados, no como predicciones ni como recomendaciones de inversión. Esta delimitación forma parte del diseño del MVP y debe mantenerse visible tanto en la memoria como en cualquier interfaz futura.

El prototipo calcula métricas como crecimiento, retornos, volatilidad, drawdown e indicadores técnicos básicos. No realiza forecasting, no recomienda comprar o vender activos y no adapta decisiones a perfiles personales de riesgo.

9.2. Riesgos de interpretación

Un riesgo relevante es que el usuario confunda una explicación descriptiva con asesoramiento financiero. Para reducirlo, el sistema debe utilizar formulaciones prudentes, explicitar el periodo analizado y recordar que los resultados históricos no garantizan comportamientos futuros.

La capa LLM aumenta este riesgo si redacta respuestas demasiado persuasivas. Por ello, la configuración actual solo permite al modelo interpretar resultados ya calculados y conserva mecanismos de fallback programático.

9.3. Seguridad en la generación y ejecución de código

La generación automática de código introduce riesgos específicos. Un script mal generado podría fallar, calcular una métrica incorrecta o acceder a recursos no previs-

tos. En este proyecto, el riesgo se reduce mediante plantillas deterministas, entradas estructuradas, ejecución controlada y captura explícita de salidas.

El código generado se ejecuta como proceso externo, con registro de salida estándar, salida de error y código de retorno. Esta estrategia permite revisar errores y evita que la respuesta final oculte fallos de ejecución.

9.4. Privacidad y datos

Los datos utilizados son series históricas de mercado y no contienen datos personales. Aun así, las claves de API de proveedores LLM se consideran información sensible y se almacenan exclusivamente en el fichero local `.env`, excluido del control de versiones.

En una versión productiva, sería necesario revisar las condiciones de uso de los proveedores de datos y de modelos, así como las políticas de retención de información enviada a APIs externas.

9.5. Sostenibilidad, costes y recursos

El enfoque incremental reduce costes computacionales. El MVP no entrena modelos desde cero y puede ejecutarse en CPU sobre datos ya descargados. La capa LLM se activa solo cuando aporta valor, principalmente en interpretación, y el modo determinista permite trabajar sin consumo de APIs externas.

Esta estrategia también mejora la sostenibilidad del desarrollo: evita ejecuciones innecesarias de modelos grandes, reduce dependencia de infraestructura GPU y permite comparar proveedores antes de elegir una configuración de uso continuado.

9.6. Limitaciones de uso

El sistema no debe utilizarse como herramienta profesional de decisión financiera sin validaciones adicionales. Su valor actual es académico y experimental: demostrar una arquitectura modular, trazable y robusta para automatizar análisis históricos.

Entre las limitaciones principales se encuentran el uso de un conjunto de datos acotado, la dependencia de datos obtenidos desde Yahoo Finance mediante `yfinance` [23], [24], la ausencia de validación financiera profesional y la no incorporación de información fundamental, noticias o variables macroeconómicas.

Capítulo 10

Conclusiones y trabajo futuro

10.1. Cumplimiento de objetivos

El trabajo desarrollado demuestra la viabilidad de un sistema multiagente para análisis financiero histórico basado en una arquitectura modular, trazable y extensible. El MVP actual completa el flujo de extremo a extremo para seis intenciones analíticas, gestiona errores básicos de entrada y conserva una base determinista reproducible.

Los objetivos planteados se cumplen en una primera versión funcional: se ha definido un workflow modular, se han implementado familias de agentes, se ha generado y ejecutado código Python de forma controlada, se han incorporado pruebas automatizadas y se ha preparado una integración controlada con modelos de lenguaje.

10.2. Aportaciones principales

La primera aportación del trabajo es la separación clara entre cálculo financiero e interpretación textual. El sistema no delega las métricas en un LLM, sino que ejecuta código controlado y utiliza el modelo, cuando está activo, para mejorar la explicación final.

La segunda aportación es la estrategia de fallback determinista. Gracias a ella, el MVP puede seguir funcionando aunque fallen las APIs externas, existan límites de cuota o se produzcan errores de red.

La tercera aportación es la validación comparativa de distintas configuraciones LLM. La memoria distingue proveedores e infraestructura: Groq se evalúa con `llama-3.3-70b-versatile` [2], [3], [4]; Gemini, con `gemini-2.5-flash` [5], [6]; y el servidor vLLM, con `openai/gpt-oss-20b` [7]. Esta separación demuestra que la arquitectura no está acoplada a un proveedor concreto.

10.3. Limitaciones actuales

El prototipo trabaja con un conjunto acotado de intenciones y con datos históricos ya descargados. No aborda predicción futura, recomendación de inversión, análisis fundamental ni incorporación de noticias. Tampoco se ha activado todavía la generación de código mediante LLM en la evaluación principal, por considerarse una fase de mayor riesgo.

Otra limitación es que la evaluación se basa en casos representativos y pruebas automatizadas, pero aún no incluye una evaluación extensa con usuarios ni una comparación cuantitativa frente a plataformas financieras profesionales.

10.4. Líneas de trabajo futuro

Como trabajo futuro se plantea ampliar el catálogo de consultas, incorporar más activos y fijar rangos temporales completamente reproducibles. También sería interesante añadir visualizaciones al informe final, mejorar la capa de logging y extender las pruebas de robustez.

En la parte LLM, el siguiente paso razonable es mantener la interpretación asistida y estudiar de forma controlada la activación de la planificación. La generación de código mediante modelo debería evaluarse solo después de definir contratos estrictos, validadores automáticos y restricciones de seguridad adicionales.

Finalmente, una evolución natural del proyecto sería construir una interfaz de usuario o API local que permita ejecutar el sistema de forma interactiva, manteniendo siempre visibles las limitaciones del análisis histórico y la ausencia de recomendación de inversión.

Referencias

- [1] Analytics Vidhya, *60+ Generative AI Terms You Must Know*, 2024. visitado 25 de mayo de 2026. dirección: <https://www.analyticsvidhya.com/blog/2024/01/generative-ai-terms/>
- [2] Groq, *Llama 3.3 70B Model Documentation*, 2026. dirección: <https://console.groq.com/docs/model/llama-3.3-70b-versatile>
- [3] Meta, *Llama 3.3 70B Instruct Model Card*, 2024. dirección: https://github.com/meta-llama/llama-models/blob/main/models/llama3_3/MODEL_CARD.md
- [4] AI@Meta, «The Llama 3 Herd of Models,» *arXiv preprint arXiv:2407.21783*, 2024. DOI: 10.48550/arXiv.2407.21783 arXiv: 2407.21783 [cs.AI]. dirección: <https://arxiv.org/abs/2407.21783>
- [5] Google Cloud, *Gemini 2.5 Flash Model Documentation*, 2025. dirección: <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-5-flash>
- [6] Gemini Team, Google, «Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities,» Google, inf. téc., 2025. dirección: https://storage.googleapis.com/deepmind-media/gemini/gemini_v2_5_report.pdf
- [7] OpenAI, *gpt-oss-120b & gpt-oss-20b Model Card*, 2025. DOI: 10.48550/arXiv.2508.10925 arXiv: 2508.10925 [cs.CL]. dirección: <https://arxiv.org/abs/2508.10925>
- [8] D. Araci, «FinBERT: Financial Sentiment Analysis with Pre-trained Language Models,» *arXiv preprint arXiv:1908.10063*, 2019. DOI: 10.48550/arXiv.1908.10063 arXiv: 1908.10063 [cs.CL]. dirección: <https://arxiv.org/abs/1908.10063>
- [9] S. Wu et al., «BloombergGPT: A Large Language Model for Finance,» *arXiv preprint arXiv:2303.17564*, 2023. DOI: 10.48550/arXiv.2303.17564 arXiv: 2303.17564 [cs.LG]. dirección: <https://arxiv.org/abs/2303.17564>

- [10] H. Yang, X.-Y. Liu y C. D. Wang, «FinGPT: Open-Source Financial Large Language Models,» *arXiv preprint arXiv:2306.06031*, 2023. DOI: 10.48550/arXiv.2306.06031 arXiv: 2306.06031 [q-fin.CP]. dirección: <https://arxiv.org/abs/2306.06031>
- [11] T. Zhou, P. Wang, Y. Wu y H. Yang, «FinRobot: AI Agent for Equity Research and Valuation with Large Language Models,» *arXiv preprint arXiv:2411.08804*, 2024. DOI: 10.48550/arXiv.2411.08804 arXiv: 2411.08804 [q-fin.CP]. dirección: <https://arxiv.org/abs/2411.08804>
- [12] Y. Xiao, E. Sun, D. Luo y W. Wang, «TradingAgents: Multi-Agents LLM Financial Trading Framework,» *arXiv preprint arXiv:2412.20138*, 2024. DOI: 10.48550/arXiv.2412.20138 arXiv: 2412.20138 [q-fin.TR]. dirección: <https://arxiv.org/abs/2412.20138>
- [13] S. Yao et al., «ReAct: Synergizing Reasoning and Acting in Language Models,» en *International Conference on Learning Representations*, 2023. arXiv: 2210.03629 [cs.CL]. dirección: <https://arxiv.org/abs/2210.03629>
- [14] T. Schick et al., «Toolformer: Language Models Can Teach Themselves to Use Tools,» *arXiv preprint arXiv:2302.04761*, 2023. DOI: 10.48550/arXiv.2302.04761 arXiv: 2302.04761 [cs.CL]. dirección: <https://arxiv.org/abs/2302.04761>
- [15] L. Wang et al., «A Survey on Large Language Model based Autonomous Agents,» *arXiv preprint arXiv:2308.11432*, 2023. DOI: 10.48550/arXiv.2308.11432 arXiv: 2308.11432 [cs.AI]. dirección: <https://arxiv.org/abs/2308.11432>
- [16] OpenAI, *OpenAI o1 System Card*, 2024. DOI: 10.48550/arXiv.2412.16720 arXiv: 2412.16720 [cs.AI]. dirección: <https://arxiv.org/abs/2412.16720>
- [17] DeepSeek-AI, «DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,» *arXiv preprint arXiv:2501.12948*, 2025. DOI: 10.48550/arXiv.2501.12948 arXiv: 2501.12948 [cs.CL]. dirección: <https://arxiv.org/abs/2501.12948>
- [18] LangChain, «LangGraph: Building Stateful, Multi-Actor Applications with LLMs,» 2024. dirección: <https://langchain-ai.github.io/langgraph/>
- [19] M. Chen et al., «Evaluating Large Language Models Trained on Code,» *arXiv preprint arXiv:2107.03374*, 2021. DOI: 10.48550/arXiv.2107.03374 arXiv: 2107.03374 [cs.LG]. dirección: <https://arxiv.org/abs/2107.03374>
- [20] Y. Lai et al., «DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation,» *arXiv preprint arXiv:2211.11501*, 2022. DOI: 10.48550/arXiv.2211.11501 arXiv: 2211.11501 [cs.SE]. dirección: <https://arxiv.org/abs/2211.11501>

- [21] Z. Ji et al., «Survey of Hallucination in Natural Language Generation,» *ACM Computing Surveys*, 2022. DOI: 10.1145/3571730 arXiv: 2202.03629 [cs.CL]. dirección: <https://arxiv.org/abs/2202.03629>
- [22] P. Lewis et al., «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,» en *Advances in Neural Information Processing Systems*, 2020. arXiv: 2005.11401 [cs.CL]. dirección: <https://arxiv.org/abs/2005.11401>
- [23] R. Aroussi, *yfinance Python Library*, 2026. visitado 25 de mayo de 2026. dirección: <https://github.com/ranaroussi/yfinance>
- [24] Yahoo Finance Help, *Download historical data in Yahoo Finance*, 2026. visitado 25 de mayo de 2026. dirección: <https://help.yahoo.com/kb/finance/historical-prices-adjusted-close-sln2311.html>
- [25] Groq, *Groq API Documentation*, 2026. dirección: <https://console.groq.com/docs>
- [26] Google AI for Developers, *Gemini API Documentation*, 2026. dirección: <https://ai.google.dev/gemini-api/docs>

Apéndice A

Anexos

A.1. Ejecución del MVP

El proyecto puede ejecutarse en modo determinista mediante los ejemplos incluidos en el repositorio:

```
python run_mvp.py --example growth_nvda
python run_mvp.py --example compare_nvda_amd
python run_mvp.py --example overview_aapl
python run_mvp.py --example returns_qqq_spy
python run_mvp.py --example risk_qqq_spy
python run_mvp.py --example technical_aapl
```

A.2. Configuración de APIs

Las claves de API se guardan localmente en el fichero `.env`, que no debe subirse al repositorio. Para trabajar sin APIs externas:

```
LLM_ENABLED=false
```

Para activar Groq, el modelo configurado es:

```
llama-3.3-70b-versatile [2], [3], [4]
```

```
LLM_ENABLED=true
```

```
LLM_PROFILE=groq
```

```
GROQ_MODEL=llama-3.3-70b-versatile
```

Para activar Gemini, el modelo configurado es:

```
gemini-2.5-flash [5], [6]
```

```
LLM_ENABLED=true  
LLM_PROFILE=gemini  
GEMINI_MODEL=gemini-2.5-flash
```

Para activar el perfil del servidor vLLM, el modelo configurado es:

```
openai/gpt-oss-20b [7]
```

```
LLM_ENABLED=true  
LLM_PROFILE=university  
UNIVERSITY_BASE_URL=https://w1.etsisi.upm.es/vllm/v1  
UNIVERSITY_MODEL=openai/gpt-oss-20b
```

A.3. Documentación interna de apoyo

Durante el desarrollo se conservaron notas técnicas, informes de EDA y fichas de evaluación en formato TXT. Estos documentos no sustituyen a la memoria, pero sirvieron como evidencias auxiliares para reconstruir decisiones de diseño y resultados experimentales. En particular, se utilizaron las notas de iteración del MVP, el informe de EDA, las fichas de evaluación determinista y LLM, la primera ejecución comparativa completa y la guía de credenciales para reforzar las secciones de metodología, datos, implementación, integración LLM y resultados.